AI Agent System Design

From Classical Computer Engineering to Modern Agent Architectures - Part I: Chapters 1-3

Working Draft v0.2 - 2026-06-29

Preface

This document is not an API tutorial and not a transcript of a conversation. It is a technical essay written from the perspective of software engineering and distributed systems. Its goal is to explain why modern AI agents increasingly resemble classical computer systems.

The central thesis is simple: LLMs were the breakthrough, but once LLMs are placed inside real products and workflows, the hard problems start to look familiar again. We need to manage state, reduce expensive remote calls, decide what to load into context, route requests to different compute layers, and design systems that remain observable, reliable, and scalable.

Part I completes the first three chapters. Chapter 1 establishes the system-level viewpoint. Chapter 2 reframes the LLM as a compute engine. Chapter 3 explains why an agent is better understood as an orchestrator than as a chatbot.

> *Core thesis: the LLM is a new compute engine, while the agent is the orchestrator that manages context, tools, state recovery and resource scheduling around it.*

Table of Contents

	Idempotency, State Machines and Replay	
Chapter 12	Multi-Agent, Concurrent Scheduling and Multi-Tenancy	Planned
Chapter 13	Agent Security: Prompt Injection, Sandboxes and Capability Boundaries	Planned
Chapter 14	Toward an Agent Operating System	Planned

Terminology

Term	Meaning in this document	Engineering analogy
LLM	Large language model used for inference	Compute Engine
Agent	System layer that organizes memory, tools, planning and context	Orchestrator / Runtime
Memory	Long-term preferences, project context and task state	Database / Cache
Context	The working set sent to the model for the current request	Working Set / Buffer Pool
Tool	External capability such as files, mail, calendar, GitHub or shell	RPC / API
Distillation	Moving capability from a large model to a smaller model or fixed workflow	Precomputation / Tiered Compute
Sandbox	An isolated environment that limits an agent's tool, file, network and code execution privileges	OS Process / Container
Idempotency	An execution constraint that makes retries safe from duplicate side effects	Payment Idempotency / Exactly-once Boundary
Replay	Reconstructing agent execution, tool calls and state transitions for debugging, audit and reconciliation	Event Log / Audit Trail

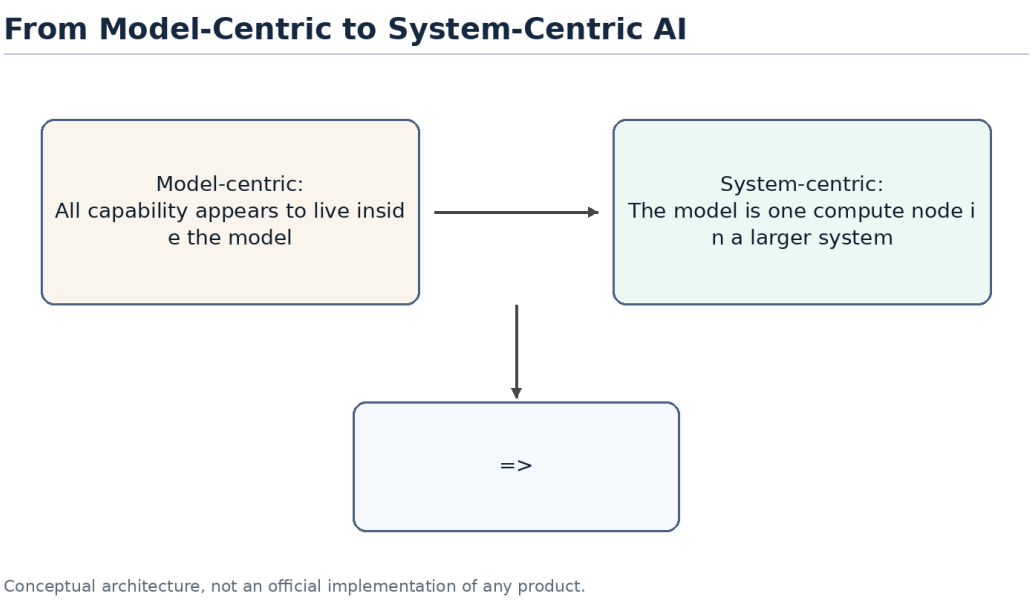

Figure 1

Figure 1. The discussion is shifting from model-centric AI to system-centric agent design.

Chapter 1. Why AI Agents Remind Me of Classical Computer Engineering

Chapter focus: establish the shift from model capability to system architecture.

1.1 The Question

Most public discussions of AI still focus on models: larger models, stronger reasoning, better coding, longer context, and better benchmarks. These improvements matter, but they do not fully explain the recent shift from chatbots to agents. The important change is that AI products are moving from answering questions to completing tasks.

Once the goal becomes task completion, the problem is no longer only model quality. The system must remember long-term state, access external tools, understand the current workspace, decide which historical information matters, and balance cost, latency and reliability. These are not new problems. They are the same type of problems classical computer engineering has been solving for decades.

1.2 Core Thesis

The core thesis of this chapter is that AI agents do not make software engineering obsolete. They make many classical software engineering ideas important again. LLMs introduce a new high-capability compute node, but building reliable, scalable and cost-efficient systems around that node still depends on layering, abstraction, caching, indexing, scheduling, statelessness, and separation of compute and storage.

From this perspective, an agent is not just a smarter chat box. It is closer to an application runtime that organizes user intent, long-term memory, project files, tool calls and model inference into a complete execution process.

1.3 Classical Engineering Concepts Behind the Pattern

Classical computer systems rarely place every responsibility inside a single component. Web systems separate application logic, cache, database, queues, object storage and external APIs. Distributed systems emphasize stateless services, externalized state, retries and horizontal scalability. Database systems emphasize indexes, query optimization, buffer pools and execution plans.

The shared goal behind these techniques is to avoid wasting expensive resources. Databases avoid full table scans. Distributed systems avoid unnecessary cross-service calls. Operating systems avoid unnecessary disk access. Modern agents face a similar problem: do not send every memory, every document and every tool result to the LLM; do not route every task to the most expensive model; do not ask the model to repeat work that can be handled by rules, small models or cached results.

1.4 How This Appears in AI Agents

In the agent world, these engineering ideas appear under new names: Memory, RAG, Context Engineering, Tool Calling, Planner, Router, Distillation, MCP and Multi-Agent systems. They sound new, and some details are new, but many system motivations are familiar.

Memory acts like external state. The context window is the working set of a request. RAG and retrieval resemble loading relevant pages. AGENTS.md can behave like a project-level prompt index. A planner resembles a scheduler. Tool calling resembles RPC. MCP resembles a tool protocol. Distillation and small models resemble moving expensive computation to a lower-cost compute layer.

1.5 Where the Analogy Breaks

Analogies help us think, but they are not proofs. AI agents differ from classical systems in important ways. Traditional APIs are usually deterministic; LLM outputs are probabilistic. A cache hit returns a fixed value; a small model generates an answer and can still be wrong. A database query has a schema; natural language tasks are often ambiguous and open-ended.

Therefore, this document does not claim that an agent is literally a database or that an LLM is literally a CPU. The better claim is that once LLMs become a new compute primitive, many proven computer engineering principles are being reapplied to AI systems.

1.6 Engineering Analysis

From an engineering perspective, the core challenge of AI agents is not only improving a single model response. The challenge is making the whole system complete tasks reliably, continuously and at reasonable cost. This requires three capabilities: context management, resource scheduling and state management.

Context management decides what information enters the model. Resource scheduling decides whether to use a rule, a small model or a large model. State management decides what should be written back to memory, files or external systems. These are system design problems, not just prompt-writing problems.

1.7 Summary

This chapter establishes the document's basic viewpoint: the rapid development of AI agents is not only the result of stronger models. It is the result of combining model capability with classical computer engineering ideas. The LLM is a new high-capability compute node, but the agent system must manage context, state, tools and cost around it.

1.8 Quick Mapping Between Agents and Classical Computer Engineering

Agent concept	Classical engineering concept	Explanation
LLM	Compute Engine	Performs high-capability inference

		but should not own all state
Memory	Database / Cache	Stores long-term preferences, project context and task history
Context Window	Working Set / Buffer Pool	The data loaded for the current request
AGENTS.md	Index / Router	A small entry point that helps locate relevant project knowledge
Tool Calling	RPC / API	Calls external software systems to perform real operations
Planner	Scheduler / Workflow Engine	Breaks tasks into steps and orders execution
Small Model	Tiered Compute	Handles common tasks cheaply and escalates complex tasks

Note: these mappings are thinking tools, not exact equivalences.

Chapter 2. LLM as a New Compute Engine

Chapter focus: position the LLM as a compute node, not as the entire system.

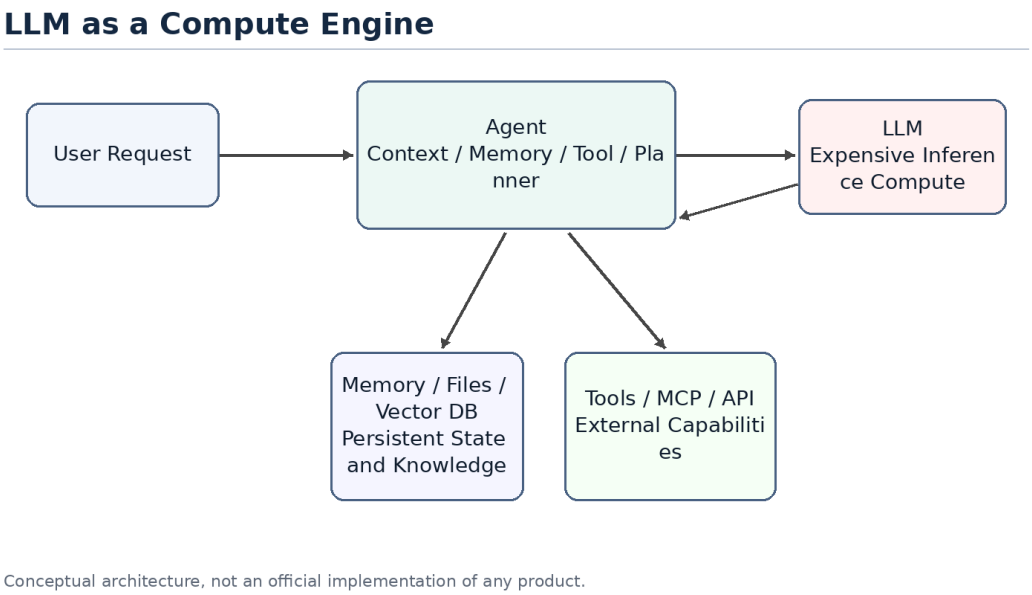

Figure 2

Figure 2. LLM as an expensive compute engine inside an agent system.

2.1 The Question

People often mix products such as ChatGPT, Claude or Gemini with the underlying LLM. A product is not just a model. It usually includes a user interface, an agent layer, memory, tool calling, permissions, file handling, monitoring, billing and safety policies. The LLM is only the most important and often the most expensive compute node inside the larger system.

Thinking of the LLM as a compute engine helps clarify agent architecture. The model does not inherently store your long-term memory or your project history. It performs inference based on the context provided in the current request. The agent layer is responsible for recovering state, selecting information and calling tools.

2.2 System Role of the LLM

In classical systems, expensive compute services are rarely exposed directly to all business logic without a management layer. Search engines have indexing services. Databases have optimizers. GPU inference clusters may have batching, caching and routing layers in front of them.

LLMs should be understood in a similar way. They can process language, code, reasoning and planning, but that does not mean all state should live inside the model or every task should go

directly to the largest model. An LLM is a powerful but expensive remote compute service. Every call consumes input tokens, output tokens, latency and GPU resources.

2.3 Stateless Compute

From the perspective of a single inference request, an LLM can mostly be treated as stateless compute. It does not permanently remember a specific user in its weights just because it had a conversation with that user in the previous turn. If the next request does not include the relevant context, the model cannot reliably know what happened before.

This means that when a product appears to remember you, the memory usually comes from the agent layer. The agent retrieves memory, recent conversation, file snippets and tool results, then composes them into the prompt. The model appears to remember because the required state has been restored for that request.

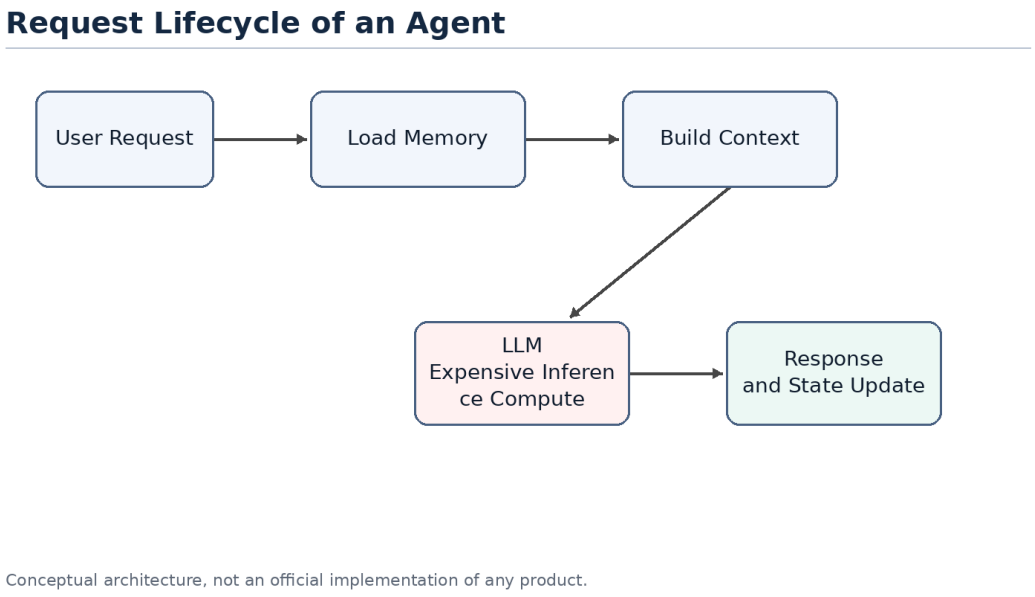

Figure 3

Figure 3. A stateless LLM requires the agent to restore context for each request.

2.4 LLM API as an Expensive Remote Call

Calling an LLM API is similar to calling an expensive remote service. It is slower than a local function call, more expensive than most database reads, and less deterministic than traditional APIs. Therefore, a central goal of agent design is reducing unnecessary calls to large models.

This mirrors classical system optimization. We used to reduce cross-service RPC calls, database queries and disk IO. Now we also reduce unnecessary LLM calls, redundant context and repeated reasoning. Tokens, context length, model tier and call count are becoming new performance metrics for AI systems.

2.5 Token Cost and Compute Cost Are Not the Same

It is important to distinguish token count from compute cost. This distinction is easy to get wrong. For example, people sometimes say that they want to use distillation to reduce token usage. Strictly speaking, distillation usually does not reduce the number of tokens. It reduces the compute cost of processing the same tokens. A small model and a large model may receive the same number of tokens, but the small model has a lower per-token cost.

The cost of an agent model call can be decomposed with a simple formula:

```
total cost ~= call count x tokens per call x cost per token
```

These three factors map to different optimization families. Planner limits, cache hits and task merging reduce call count. Context engineering, retrieval, pruning, summarization, prompt caching and compressed tool outputs reduce tokens per call. Distillation, small models, routing and cascades reduce cost per token.

Dimension	**Reducing Token Count**	**Reducing Per-Token Compute Cost**
Optimization target	Number of input and output tokens per request	Compute and price required to process the same tokens
Distributed-systems analogy	Reduce RPCs, shrink payloads and reduce round trips	Move work to a cheaper compute tier, similar to hot/cold tiering or service degradation
Typical techniques	Context engineering, RAG, pruning, summarization, prompt caching, compressed tool outputs and planner iteration limits	Distillation, small models, routing and cascades, where a cheap model tries first and uncertain cases escalate
Need for training/data	Usually no; engineering and configuration can be enough	Distillation needs labeling, training, evaluation and deployment; routing or existing small models may not
Implementation cost and speed	Low cost and fast feedback; prompt, retrieval and cache changes can help immediately	Distillation is high cost; routing is medium cost; benefits depend on stable tasks and evaluation
Main risk	Over-pruning loses context and makes the model answer from incomplete information	Wrong-tier routing: simple tasks hit large models and waste money, or hard tasks are sent to weak models and fail
Source of uncertainty	Retrieval recall, summarization fidelity and whether tool-output compression damages information	Small-model generalization boundaries and confidence estimation quality
Priority for independent developers	Do this first; low barrier and usually no training pipeline	Consider later, after traffic and data become stable; distillation is not the starting move

Therefore, a more precise statement is that distillation primarily reduces the use of expensive large models, not necessarily the number of tokens. Token usage decreases only when distillation shortens the workflow, reduces repeated planning calls, or replaces several large-model calls with a smaller computation. In that case, the saved tokens come from removed calls and intermediate context, not from distillation directly compressing the tokens inside one request.

This distinction matters in real products. For an independent developer, distillation is a heavy tool: it requires labeled data, a training pipeline, evaluation sets, deployment and model hosting. The token-reduction family looks more like ordinary systems optimization: cache stable prefixes, prune irrelevant context, compress tool outputs, limit planner iterations and avoid unnecessary large-model calls through routing. These can usually work without training.

So if the goal is to reduce cost first, the practical order is usually to exhaust call-count and token-count reductions before moving to routing, small models and distillation. Distillation belongs later, especially for workflows with stable traffic, stable task distributions and enough examples.

2.6 Analogy to Classical Compute

As a compute engine, an LLM can be compared to a database execution engine, a remote compute service or a GPU inference cluster. It is powerful, but it should not carry every responsibility in the system. A database does not own business workflow. A CPU does not perform operating-system scheduling by itself. Similarly, an LLM should not be viewed as the entire agent system.

This viewpoint explains why the agent layer matters. The agent does not replace the model; it ensures that the model is called at the right time, with the right context, and at a reasonable cost.

2.7 Summary

This chapter positions the LLM as a new high-capability compute engine. It is powerful, expensive, mostly stateless and usually accessed through an API. This explains why memory, context, tools and planning should be viewed as parts of the agent system rather than as properties of the model alone.

Chapter 3. Agent as an Orchestrator

Chapter focus: the agent is the orchestration layer that routes work to the right resource.

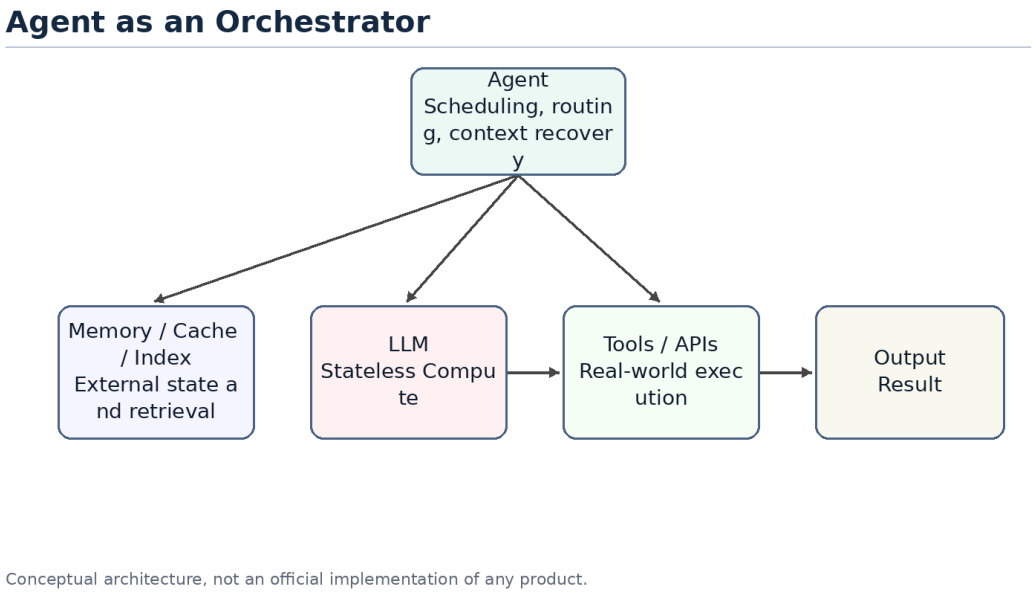

Figure 4

Figure 4. The agent as an orchestrator rather than a chatbot.

3.1 The Question

If the LLM is the compute engine, what is the agent? A common misunderstanding is to treat an agent as a wrapper around a large model or as a chatbot with tools. That description captures part of the surface behavior, but it misses the architectural role.

A better definition is that an agent is an orchestrator. It receives a user goal, restores relevant state, selects context, decides whether to call tools, chooses which model or compute layer to use, and organizes multiple steps into an executable workflow.

3.2 Core Components of an Agent

A typical agent contains several logical components. Memory stores long-term preferences, project background and task history. A context builder decides what should enter the current prompt. A planner decomposes the user goal into steps. A tool layer calls external systems. A router or scheduler decides whether to use rules, small models or large models. An evaluator or guardrail decides whether the result is reliable enough.

These components may be physically distributed across multiple services or hidden inside a product. But logically, they are part of the agent layer, not part of the LLM itself.

3.3 Typical Execution Flow

When an agent receives a request, it should not immediately send the raw user message to the largest model. A more efficient flow is to classify the task, retrieve relevant memory or files, build context, then select a compute resource. If a simple rule can solve the task, no model is needed. If the task matches a common pattern, a small model may be enough. If the task is complex or risky, the agent escalates to a large model.

This is the difference between a chatbot and an agent. A chatbot tends to map one input to one response. An agent receives a goal and organizes a set of computations and operations to complete it.

3.4 Agent, API Gateway and Scheduler

From a systems perspective, an agent resembles a mixture of API gateway, workflow engine and scheduler. It exposes a unified interface to the user while connecting internally to models, tools, memory, files and external services. It also chooses an execution path based on task complexity and cost.

That is the role of an orchestrator. It may not perform all computation by itself, but it decides which resources participate, in what order they participate, how results are combined, and whether failures should trigger retries, fallbacks or escalation.

3.5 Tiered Compute: Rule, Cache, Small Model, Large Model

Agent scheduling can be abstracted as tiered compute. The lowest layer is rules: lowest cost and fastest, but limited coverage. The next layer is cache or retrieval, which reuses existing results or finds relevant context. Above that is a small model. A small model does not store fixed answers like a cache; it stores capabilities learned through training or distillation. The highest layer is the large model, which is the most capable and the most expensive.

This resembles multi-level caches, hot/cold storage tiers and service degradation strategies in traditional systems. A good agent should not route every request directly to the most expensive model. It should solve as many requests as possible at cheaper layers.

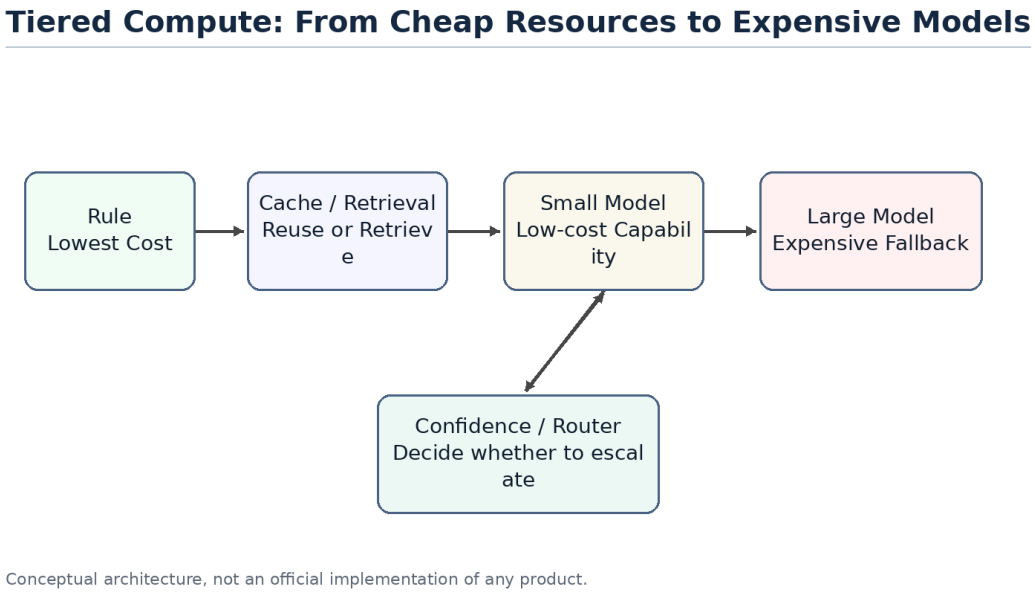

Figure 5

Figure 5. Tiered compute prevents every request from hitting the largest model.

3.6 Why a Small Model Is Not Just a Cache

Small models are sometimes compared to caches, but that analogy needs refinement. A normal cache stores results. On a hit, it returns a fixed value. A small model stores capability. It still performs inference based on the input. It cannot guarantee a fixed answer like Redis, but it can generalize to problems that were not explicitly seen during training.

A better phrase is capability cache. Distillation compresses behavior observed from a larger model into a lower-cost compute layer. It does not eliminate computation; it moves computation from an expensive model to a cheaper model.

3.7 Risks and Limits of Agent Orchestration

An agent as orchestrator introduces new risks. A bad router may send simple tasks to an expensive model or complex tasks to an underpowered model. Bad memory retrieval can make the model reason over the wrong context. Multi-step tool calls can amplify small mistakes. Probabilistic model behavior makes traditional testing incomplete.

Therefore, agent systems need observability, logs, audits, replay, evaluation sets and escalation policies. This again shows that agents are not just conversation interfaces. They are software systems that need system engineering discipline.

3.8 Summary

This chapter positions the agent as an orchestrator. It is not the model itself. It is the system layer that organizes memory, tools, planning, context and resource scheduling around the

model. This prepares the ground for later chapters: why memory resembles storage, why AGENTS.md resembles an index, why distillation resembles tiered compute, and why multi-agent systems increasingly resemble distributed systems.

Plan for the Remaining Chapters

Part I completes the foundational system perspective. Later chapters will continue the same line of reasoning: how memory maps to storage, how stateless agents map to stateless services, why context engineering resembles query optimization, why AGENTS.md can act like an index, and why distillation and small models fit naturally into tiered compute.

The next stage should also add three engineering themes that strengthen the book's differentiation. First, agent security should not remain a footnote. Operating-system security models will migrate into agent systems: prompt injection behaves more like an exploit, tool use needs capability boundaries, and code or external-system access needs sandboxing. Second, concurrent scheduling is where the OS analogy becomes most literal. Multi-agent, multi-tenant execution, task queues and resource isolation go beyond the single-task Planner-to-Scheduler analogy. Third, agents should be treated as production systems: they need SLAs, idempotency, optimistic locking, state machines, retries, degradation, escalation, observability, audit, replay and reconciliation. This production reliability perspective is the main difference from OS-analogy papers that focus mostly on architecture and runnable prototypes.

1. 1. Chapter 4 will separate the responsibilities of memory, tools and planner.
2. 1. Chapters 5-6 will focus on compute/storage separation and stateless agents.
3. 1. Chapters 7-8 will connect context engineering and AGENTS.md with database optimization.
4. 1. Chapters 9-10 will discuss retrieval, token reduction, distillation, small models and tiered compute, with emphasis on separating token-count reduction from per-token compute-cost reduction.
5. 1. Chapter 11 will treat agents as production systems, focusing on idempotency, optimistic locking, state machines, retries, degradation, escalation, observability, audit, replay and reconciliation.
6. 1. Chapter 12 will cover multi-agent, multi-tenant and concurrent scheduling, moving the Planner-to-Scheduler analogy from single-task execution to resource contention and orchestration.
7. 1. Chapter 13 will add the missing agent security model, including prompt injection as exploit, tool capability boundaries, sandboxing, least privilege and audit.
8. 1. Chapter 14 will return to the agent operating system idea and connect memory, tools, planning, security, scheduling and production reliability into one system view.